

UTEM - Ingeniería Civil en Ciencia de Datos

UNIVERSIDAD TÉCNICA METROPOLITANA

Facultad de Ingeniería

Departamento de Informática y Computación

Evaluación de Infraestructura y Procesamiento Paralelo

Análisis práctico desde los fundamentos del hardware hasta la gestión de datos distribuidos

Estudiante:	dylan novoa
Asignatura:	INFB6074 - INFRAESTRUCTURA PARA C. DE DATOS
Evaluación:	Semanas 1 a 6 (Evaluación Integradora 1)
Carrera:	Ingeniería Civil en Ciencia de Datos

RESUMEN EJECUTIVO

El presente informe recopila los resultados y análisis del desarrollo práctico correspondiente a las semanas 1 a 6 de la asignatura.

El propósito central de esta investigación experimental fue comprender cómo las características internas del hardware condicionan el rendimiento del software analítico moderno, transitando desde la programación directa en bajo nivel hasta la toma de decisiones basada en la evaluación comparativa de arquitecturas distribuidas masivas.

En la primera etapa se diseñó un programa de arranque básico (Bootloader) en ensamblador de 16 bits para interactuar de forma pura con la memoria de video e inicializar el sistema sin capas lógicas intermedias.

Posteriormente, se construyeron perfiladores personalizados en Python para registrar los

techos de rendimiento y latencia del procesador, de la memoria RAM y del disco. Mediante la comparación de accesos secuenciales y aleatorios se validó de manera empírica el impacto drástico de los fallos de caché en la velocidad general de cómputo.

Finalmente, se abordó la optimización Big Data mediante el uso de hilos independientes de ejecución, el despliegue de pipelines basados en la metodología estructurada Medallion, la interconexión de servicios relacionales bajo un ambiente multi-contenedor coordinado y, como etapa de cierre en el Ejercicio 6, el diseño de una matriz analítica multicriterio para evaluar la viabilidad de distintas infraestructuras frente a escenarios organizacionales reales.

CONTEXTO CONCEPTUAL

En las ciencias de datos actuales es habitual operar sobre herramientas de abstracción avanzada. No obstante, la correcta administración, almacenamiento e interrogación de volúmenes masivos de información exige un conocimiento riguroso de las limitaciones físicas impuestas por la infraestructura tecnológica.

Los objetivos de esta evaluación se centran en diagnosticar de forma controlada los cuellos de botella del hardware (CPU, memoria, entrada/salida y red) y justificar con base en métricas el uso de herramientas modernas de aislamiento. Esta progresión permite comprender que técnicas analíticas complejas como el almacenamiento columnar Parquet, la vectorización en memoria de DuckDB o el despliegue distribuido de servicios mediante Docker Compose no son elecciones arbitrarias, sino respuestas estructuradas ante las ineficiencias físicas inherentes al hardware tradicional.

EJERCICIO 1: DESARROLLO DE UN MINI SISTEMA BOOTABLE

Diseño del Bootloader

Para el primer ejercicio, se planteó el desarrollo de un código de arranque mínimo autónomo escrito en lenguaje Ensamblador de 16 bits (Real Mode). Este diseño obliga a interactuar directamente con la arquitectura x86 del procesador y los segmentos físicos de la memoria sin contar con los servicios estándar de un kernel comercial o sistema operativo anfitrión.

El script utiliza la directiva de compilación

`org 0x7C00`

, la cual establece el offset de memoria exacto donde el firmware de la placa madre (BIOS) carga por defecto el primer sector del disco antes de transferirle el control al procesador.

Interacción con la Pantalla y Memoria de Video

Al carecer de un controlador o consola nativa, la visualización de caracteres en pantalla se resolvió de forma manual mediante dos fases lógicas:

1. **Limpieza de interfaz:** Se limpian los registros generales de la CPU y se ejecuta la interrupción de pantalla

int 0x10 de la BIOS para resetear el modo de video a texto estandarizado.

2. **Escritura Directa en Memoria VGA:** Se configuró el registro de segmento extra para apuntar directamente a la dirección física 0xB800, que corresponde al búfer de video en modo texto. Cada letra renderizada se inyecta utilizando dos bytes contiguos: el primero almacena el código ASCII del carácter y el segundo define el byte de atributos cromáticos (establecido en 0x0B para un fondo negro con letras en Cyan Claro).
3. **Manejo del Salto de Línea:** Se implementó un algoritmo aritmético con las instrucciones `div cl` y `mul cl` que calcula el múltiplo exacto de los 160 bytes que componen una línea de pantalla completa (80 caracteres por 2 bytes cada uno), lo que permite desplazar el cursor de forma segura hacia el inicio del renglón inferior cada vez que se detecta el código ASCII 10 ("Enter").

Ensamblado y Emulación del Entorno

El código fuente se tradujo a binario plano utilizando el ensamblador NASM (versión 2.16) forzando una salida cruda bit a bit. El archivo resultante tiene un tamaño exacto de 512 bytes y culmina con la palabra mágica de arranque

0xAA55

. La validación se llevó a cabo de manera segura mediante el emulador de hardware QEMU (versión 11.0.0).

EJERCICIO 2: BENCHMARK DE COMPONENTES DEL SISTEMA

Desarrollo de un Perfilador de Rendimiento

Con el objetivo de mapear las velocidades crudas del computador anfitrión, se programó un script de evaluación analítica en Python sin recurrir a software externo. El entorno físico de evaluación consistió en una CPU de 12 núcleos lógicos, 16 GB de Memoria RAM y un disco sólido de 1 TB (con 500 GB de espacio libre durante el test).

Escenarios de Estrés Evaluados

Se definieron cuatro funciones específicas para estresar de manera independiente los subsistemas de hardware, repitiendo cada medición en tres ciclos consecutivos para promediar los resultados:

- **Cómputo Intensivo (CPU):** Generación y cálculo trigonométrico masivo (seno y coseno) sobre un vector de 10 millones de números flotantes aleatorios de NumPy, saturando las unidades aritméticas de la CPU.
- **Acceso Secuencial a Memoria:** Sumatoria consecutiva sobre un arreglo ordenado de 10 millones de enteros, maximizando el acierto de pre-búsqueda (prefetching) de las memorias caché L1/L2.

- **Acceso Aleatorio a Memoria:** Sumatoria desordenada saltando caóticamente entre posiciones mediante índices aleatorios. Esta prueba degrada la localidad de los datos, provocando fallos de caché continuos (cache misses).
- **I/O de Almacenamiento (Disco):** Escritura y lectura síncrona de bloques binarios de 10 MB generados aleatoriamente con la función `os.urandom`.

Discusión de Cuellos de Botella

Los datos registrados y analizados mediante Pandas y matplotlib demostraron con precisión matemática la penalización por localidad. El acceso secuencial a memoria fue órdenes de magnitud más rápido que el acceso aleatorio, evidenciando el tiempo muerto que pierde el procesador al verse forzado a buscar datos en la RAM principal cuando no se encuentran en la caché. Asimismo, las llamadas síncronas de lectura y escritura en el almacenamiento persistente demostraron ser el eslabón más lento del flujo, justificando por qué las canalizaciones de datos masivos intentan priorizar operaciones vectorizadas directamente en memoria.

EJERCICIO 3: PROCESAMIENTO PARALELO Y MEDICIÓN DE SPEEDUP

Descripción del Escenario Experimental

Para el tercer ejercicio, se evaluaron los beneficios del procesamiento concurrente local sobre un conjunto simulado de 500,000 registros de transacciones. La tarea asignada incluyó el cálculo de la media, la varianza y la métrica de desviación estándar (Z-Score) por cada bloque, sumado a una carga artificial de cómputo flotante trigonométrico para inducir un consumo de tiempo controlable en la CPU.

Métricas de Rendimiento Aplicadas

Se comparó el tiempo de ejecución secuencial (T_1) con esquemas paralelos distribuidos en 2 y 4 subprocesos concurrentes (T_p) mediante el uso de la clase

`ProcessPoolExecutor`

de Python. Para la evaluación técnica, se aplicaron dos fórmulas de cómputo de alto rendimiento:

- **Speedup:** $Sp = T_1 / T_p$
- **Eficiencia:** $Ep = Sp / p$

Los resultados empíricos demostraron un incremento sustancial en el factor de velocidad (Speedup sublineal), aunque la eficiencia decrece moderadamente al incorporar 4 trabajadores. Esto se debe principalmente a la porción serial no paralelizable del script (Ley de Amdahl), al costo administrativo del sistema operativo al clonar procesos en memoria (fork overhead) y al tiempo consumido por el intérprete al empaquetar y desempaquetar las estructuras de datos que viajan entre los núcleos (serialización pickle).

EJERCICIO 4: PIPELINE BIG DATA CON DOCKER Y PARQUET

Estructura de la Arquitectura Medallion

Para procesar flujos masivos de eventos recolectados por telemetría IoT de un hogar inteligente, se construyó un canal de procesamiento automatizado estructurado en tres zonas lógicas para garantizar la trazabilidad del dato:

- **Zona Bronze (Datos Crudos):** Recibe de forma inmutable el archivo crudo original de sensores mediante el script de ingesta segura `bronze.py`.
- **Zona Silver (Validación y Calidad):** El módulo `plata.py` filtra el dataset aplicando reglas de negocio estrictas: elimina registros nulos en el ID, descarta filas duplicadas exactas y purga anomalías térmicas fuera de rango lógico. Una vez limpio, transforma el archivo plano a formato binario columnar Apache Parquet usando el motor PyArrow.
- **Zona Gold (Métricas de Negocio):** Mediante consultas SQL vectorizadas en el motor DuckDB, se leen los archivos Parquet directamente a velocidad de hardware nativo para generar reportes consolidados sobre perfiles climáticos por habitación, eficiencia energética (HVAC) y picos de acumulación de CO2, exportando los resultados limpios a la carpeta externa `metrics/` en formatos CSV y JSON de forma automática.

Aislamiento y Catálogo de Datos

El pipeline fue empaquetado mediante un archivo Dockerfile basado en `python:3.11-slim` con dependencias fijas en `requirements.txt`

. La inclusión de volúmenes enlazados asegura la reproducibilidad absoluta del entorno, permitiendo que el procesamiento ocurra de manera aislada pero escribiendo los resultados persistentes de vuelta en el disco de Windows de forma transparente.

EJERCICIO 5: MINI INFRAESTRUCTURA DOCKER COMPOSE

Diseño de la Arquitectura Multi-Servicio

El último ejercicio consistió en el despliegue de una mini infraestructura analítica coordinada mediante Docker Compose. El ecosistema interconecta tres contenedores aislados que corren en una red virtual tipo bridge privada llamada

ds_network

, previniendo la exposición pública de puertos y garantizando comunicación interna mediante resolución DNS nativa de Docker:

1. **PostgreSQL (Motor Relacional):** Inicializa automáticamente las tablas estructuradas de ventas y almacenamiento analítico flexible mediante el script 01_crear_tablas.sql. Incluye una validación de salud (healthcheck) mediante la herramienta pg_isready.
2. **Jupyter Lab (Capa de Cómputo):** Entorno analítico personalizado que arranca de forma condicional mediante depends_on una vez que el motor relacional se encuentra completamente saludable, evitando condiciones de carrera de conexión.
3. **pgAdmin (Interfaz Gráfica Web):** Herramienta visual para la administración remota de las bases de datos del esquema.

Persistencia y Flujo de Análisis

El flujo se validó ejecutando dos scripts automatizados desde Jupyter. El script

01_cargar_datos.py

generó un dataset de 500 ventas simuladas, persistió una copia en formato CSV y lo cargó masivamente en la base de datos PostgreSQL. Posteriormente, el script

02_analisis.py

interrogó al motor mediante consultas analíticas avanzadas (funciones de ventana y agregaciones mensuales), generó gráficos visuales interpretativos y persistió los resultados del análisis de vuelta en el motor relacional en formato JSONB. El uso de volúmenes de datos con nombre (named volumes) asegura que la información resida de forma permanente en el host aun si los contenedores se detienen por completo.

CONCLUSIONES

La culminación de este marco experimental integrador ha permitido consolidar una visión sistémica de la computación aplicada a la ciencia de datos. Se ha verificado de manera medible que la optimización de los sistemas de información no responde exclusivamente a la incorporación de lenguajes o librerías sofisticadas, sino a un entendimiento riguroso del comportamiento del hardware subyacente.

El desarrollo práctico demostró que limitaciones de bajo nivel, como la latencia de almacenamiento físico y los fallos de lectura en las memorias caché, pueden ser mitigados con éxito mediante decisiones arquitectónicas conscientes. La transición estructurada desde la programación nativa en ensamblador hacia el diseño de almacenamiento columnar indexado, canalizaciones Medallion aisladas y redes multi-servicio distribuidas comprueba que el diseño reproducible y modular es el estándar fundamental para construir ecosistemas Big Data eficientes, seguros y robustos en la industria moderna.

REGISTRO TÉCNICO DEL ENTORNO

De acuerdo con las exigencias de auditoría técnica descritas en la pauta de la evaluación, se declara formalmente el inventario de hardware y software utilizado para la validación y ejecución del proyecto:

- **Sistema Operativo Host:** Windows 11 Pro de 64 bits (Entorno de desarrollo PowerShell)
- **Arquitectura del Procesador:** CPU multinúcleo con 12 hilos lógicos de procesamiento disponibles
- **Memoria RAM del Sistema:** 16 GB de memoria unificada
- **Almacenamiento Persistente:** Unidad de estado sólido de 1 TB (500 GB disponibles durante las pruebas)
- **Versión de Motor Docker:** Docker Desktop Versión 29.5.2, build 79eb04c
- **Versión de Compiladores Básicos:** NASM Versión 2.16.01 y QEMU Emulator Versión 11.0.0
- **Entorno de Lenguaje Analítico:** Python Versión 3.11 encapsulado dentro de imágenes controladas de contenedores

CONSOLIDADO DE COMANDOS DE EJECUCIÓN Y REPRODUCCIÓN

Para garantizar la reproducibilidad absoluta de las pruebas y entornos experimentales descritos desde el Ejercicio 1 hasta el Ejercicio 6, se detallan los comandos estructurados para cada fase de la evaluación:

- Ejercicio 1: Compilación y Emulación del Entorno Bootable
\$ nasm -f bin boot.asm -o boot.bin
\$ qemu-system-i386 -drive format=raw,file=boot.bin
- Ejercicio 2: Ejecución del Microbenchmark de Hardware Local
\$ python benchmark_arquitectura.py
- Ejercicio 3: Evaluación de Tiempos y Speedup Concurrente
\$ python ejercicio_03_paralelismo.py
- Ejercicio 4: Construcción e Ingesta del Pipeline en Contenedor Aislado
\$ docker build -t infb6074-pipeline .
\$ docker run --rm -v "\$(pwd)":/work infb6074-pipeline
- Ejercicio 5: Despliegue de Mini Infraestructura Multi-Servicio
\$ docker compose up --build
\$ docker compose ps
\$ docker compose logs
\$ docker compose down
- Ejercicio 6: Análisis Multicriterio de Arquitecturas Big Data
\$ python evaluar_matriz_arquitectonica.py

DECLARACIÓN DE ASISTENCIA DE INTELIGENCIA ARTIFICIAL

En cumplimiento estricto de las políticas de integridad académica de la Universidad Técnica Metropolitana, se declara que la confección de este reporte técnico y la optimización de los scripts analíticos contaron con la asistencia del modelo de inteligencia artificial generativa Gemini como herramienta interactiva de soporte. El uso de la herramienta se acotó a la estructuración del formato documental, la depuración (debugging) de errores sintácticos de compilación en los entornos aislados de Docker y la agilización del diseño estético visual, garantizando que la lógica algorítmica de los ejercicios, el análisis crítico de las métricas de rendimiento y la ejecución física de las pruebas corresponden íntegramente a un trabajo de autoría propia.

REFERENCIAS BIBLIOGRÁFICAS

- Docker Product Team. (2026). *Docker Core & Compose Virtualization Framework Documentation*. Recuperado de: <https://docs.docker.com/>
- QEMU Open Source Community. (2026). *QEMU Processor Emulator and Virtualization Reference Guide*. Recuperado de: <https://www.qemu.org/>
- Python Software Foundation. (2026). *Python Language Standard Library Manual (v3.11)*. Recuperado de: <https://www.python.org/doc/>
- Pandas Development Core Team. (2026). *Pandas DataFrames and Structured Matrices Analysis Documentation*. Recuperado de: <https://pandas.pydata.org/docs/>
- Kaggle Public Data Science Platform. (2026). *Data Repository and Smart Home IoT Benchmark Collections*. Recuperado de: <https://www.kaggle.com/>
- UCI Machine Learning Repository. (2026). *IoT Datasets and Time-Series Operational Telemetry Records*. Recuperado de: <https://archive.ics.uci.edu/>
- YouTube LLC. (2026). *Historial de reproducciones, tutoriales técnicos y conferencias de ingeniería sobre arquitecturas de bajo nivel y Docker Compose*.